Before submitting (delete this box): Fill in every highlighted *[CONFIRM: ...]* with your real details, or delete the sentence if it doesn't apply. Canonical will ask about these answers in later interviews, so every claim must be accurate. Your name, email, phone and links have been left out, as Canonical asks; keep the filename neutral as well.

Written Interview

Web Frontend Engineer – JS, CSS, React, Flutter

1. Web engineering experience

What skill or knowledge have you acquired in the past year that has been particularly helpful?

Over the past year I have gone deep into **React Native and mobile performance engineering**. I designed and built, end to end, a cross-platform crypto markets and portfolio app for iOS and Android from one TypeScript codebase. It has live market data over WebSockets, interactive price charts, portfolio tracking, and price alerts with push notifications. To make it feel native I had to learn how the JavaScript thread, the UI thread and the bridge interact. The tools I used were the Hermes engine, cold-start optimisation, list virtualisation, strict re-render control, and gesture-driven 60fps animations with Reanimated and Gesture Handler. I also automated builds and distribution with EAS and TestFlight, and covered the app with Jest and React Native Testing Library.

What motivated you to learn it?

For eight years I worked mainly in the browser. I realised that most users now reach products from a phone first, and I wanted my React knowledge to be cross-platform. I also wanted a project I owned completely: architecture, build pipeline, release and maintenance, not only one feature area of a large client codebase. Building in fintech, which I know well from payments work, let me focus on the hard engineering problems, mainly real-time data and smooth rendering under load. This interest in one codebase serving several platforms is also why Canonical's Flutter direction caught my attention.

What has the impact been for you and your team?

For me, it changed how I think about rendering cost. On mobile every unnecessary re-render is visible as dropped frames, so I measure first and optimise second. I now bring that discipline back into my web work as Dev Lead on Adobe Workfront's Document Management area: memoisation boundaries, virtualised lists for large document sets, and watching bundle size and main-thread work. I share these practices in code reviews and in mentoring the developers on my team. *[CONFIRM: add one concrete example, e.g. "I introduced list virtualisation for the document list, which cut render time from X to Y", or a talk/knowledge-sharing session you gave]*

Describe your experience of web programming – JavaScript, TypeScript, React, CSS and Python in particular.

JavaScript (9 years). JavaScript has been my main language since 2017, from ES5/ES6 on the Western Union merchant portal up to ES2024 on Adobe Workfront today. I am comfortable with the language in depth: closures, the event loop and asynchronous patterns (promises, async/await, RxJS observables), modules, and tooling such as Webpack, Babel and ESLint.

TypeScript. I use TypeScript daily. All my Angular projects (Fiserv, Cisco Webex, Hudson MX) were strictly typed, and so is my React Native app. I use types as documentation and as a safety net for API contracts, for example by modelling payment states as discriminated unions so that impossible states cannot be represented.

React. At Adobe Workfront I lead front-end development on React, Redux, React Router and React Spectrum, Adobe's accessible design system, in a large shared codebase serving enterprise customers. At Mastercard I worked with React, GraphQL/Apollo and Ant Design on a payments management suite. There I untangled long-standing technical debt and delivered critical features on a dynamic form engine that reshapes the payment form per client. I also have strong Angular experience (Angular 5 through modern versions, NgRx, RxJS), which gives me a useful comparison of the two ecosystems' approaches to state and change detection.

CSS. I write CSS by hand and care about it: SCSS, Flexbox, CSS Grid, Tailwind, responsive and component-based styling, and theming through design tokens. I have built UIs on several design systems (React Spectrum, Ant Design, PrimeNG, Bootstrap), so I understand them from the consumer side and from the side of keeping a component library consistent across teams. That is directly relevant to the Vanilla Framework. I treat accessibility as part of CSS and markup quality: semantic HTML, focus management, colour contrast and keyboard navigation.

Python. *[CONFIRM: describe honestly, e.g. "Python is not my main professional language. I used it at university for … and for small scripts/automation such as …" or "I have no production Python experience yet, but …"]* I am comfortable reading and contributing to back-end code when it helps the front end, and I would welcome building more Python services alongside the back-end engineers. I also have some PHP experience from WordPress work.

Describe your experience building large systems with many services – web front ends, REST APIs, data stores, event processing and other kinds of integration between components.

Most of my career has been building the front ends of large, multi-service systems where the UI is the place where many back-end services meet:

- **Adobe Workfront (Document Management)**: upload, preview, versioning and approval of creative assets inside enterprise workflows. These features depend on several services (storage, preview generation, versioning, permissions and workflow), and the UI has to handle long-running asynchronous operations and partial failures gracefully.
- **Mastercard**: a payments management suite with a GraphQL/Apollo data layer over payment, account, user and user-group services, plus an admin layer for payment

settings. The dynamic form engine was configuration-driven, so the front end had to stay correct for every client configuration.

- **Fiserv**: a payments application for creating, viewing and modifying internal and external payments, consuming REST APIs. I built it from the first commit: architecture, Angular workspace, core features, and the conventions the team adopted.
- **Western Union**: a merchant portal for small and medium enterprises with single and multiple points of sale, aggregating transaction and account data.
- **My React Native app**: real-time event streams over WebSockets combined with REST endpoints for history, local persistence, and push notifications for alerts.

On these projects I worked closely with back-end engineers on API contracts: response shapes, pagination, error models, and how the UI should behave on timeouts or retries. *[CONFIRM: add any direct back-end, database or event-processing work you did yourself (e.g. Node.js endpoints, SQL, queues); otherwise keep it clear that your ownership was the front end and the contract]*

What are the key things to think about in regard to architecture, maintainability, and reliability in these large systems?

- **Clear contracts between components.** Well-designed REST APIs (or GraphQL schemas) are the real architecture of a multi-service system. They should be explicit, versioned, documented and typed end to end, so the front end and back end can evolve independently.
- **Clear ownership and boundaries.** Each service and each front-end module should have one clear responsibility and owner. Shared component libraries such as a design system remove duplication without coupling teams' release cycles.
- **Designing for failure.** In distributed systems something is always failing somewhere. The UI needs explicit loading, empty, error and partial states. Back ends need timeouts, retries with backoff, and idempotent operations. In payments, idempotency is what stops a retried request from becoming a double payment.
- **Data consistency and caching.** Decide deliberately where the source of truth is, how stale data may be, and how the client cache is invalidated, especially for real-time data.
- **Observability.** Logging, metrics, tracing and front-end error reporting, so that issues are found by the team before customers report them.
- **Automated testing at the right levels.** Unit tests for logic, component tests for UI behaviour, contract tests between services, and a small set of end-to-end tests for critical user journeys.
- **Safe delivery.** CI/CD, code review, feature flags and the ability to roll back quickly. Small, frequent releases are less risky than big ones.
- **Simplicity and consistency.** Shared conventions, linting and documented decisions make a large codebase readable for the next engineer. I learned this when I set the patterns for the Fiserv application, which the rest of the team then followed.

Describe any experience you have with Flutter.

[CONFIRM: if you have any Flutter/Dart experience, describe it here. If not, keep the honest version below.] I have not yet shipped a production Flutter application. My closest experience is React Native, where I built and released a cross-platform app from a single

codebase. Many of the ideas transfer directly: declarative UI, a component (widget) tree, unidirectional state management, and the focus on 60fps rendering and startup time. I see Flutter as a natural next step for me. It goes further than React Native by owning the rendering itself, which makes consistent desktop, mobile and web targets possible from one codebase. I am ready to learn Dart and Flutter seriously and quickly, as I did with React Native.

2. Software engineering experience

What kinds of software projects have you worked on before?

I have spent nine years at Endava delivering front-end solutions for global enterprise and fintech clients:

- **Payment platforms** for Western Union (merchant portal), Fiserv (payments application built from scratch) and Mastercard (payments management suite with dynamic, per-client forms).
- **Insurance software** for Cisco Webex: complex, data-heavy interfaces for insurers modernising their operations.
- **Enterprise work management** at Adobe Workfront, where I am Dev Lead for Document Management.
- **Advertising technology** at Hudson MX: a cloud-based media-buying platform where I joined mid-project and became productive quickly in an unfamiliar codebase.
- **Mobile**: my own React Native crypto markets and portfolio app for iOS and Android.
- **Websites**: WordPress/PHP projects. *[CONFIRM: add context or delete]*

Which operating systems, development environments, languages, databases and frameworks?

Area	Experience
Operating systems	*[CONFIRM: e.g. macOS / Windows / Ubuntu Linux; mention any Ubuntu use specifically]*
Development environments	*[CONFIRM: e.g. VS Code, WebStorm]*, Git/GitHub, CI/CD pipelines, Chrome DevTools, Xcode and Android Studio for React Native
Languages	TypeScript, JavaScript (ES5–ES2024), HTML5, CSS3/SCSS, PHP *[CONFIRM: Python / others]*
Frameworks and libraries	React, React Native (Expo/EAS, Reanimated, Gesture Handler, React Navigation, Hermes), Angular, Redux / Redux Toolkit, NgRx, RxJS, GraphQL / Apollo Client, Transloco
UI systems	React Spectrum, Ant Design, PrimeNG, Bootstrap, Tailwind CSS
Testing	Jest, React Native Testing Library, Jasmine, Cypress (E2E)
Build tooling	Webpack, Babel, ESLint, EAS / TestFlight
Databases and storage	*[CONFIRM: e.g. MySQL (WordPress), PostgreSQL, client-side storage such as AsyncStorage/IndexedDB]*

Outline your thoughts on open-source software development. Have you been an open-source maintainer, and can you describe the scope of your contributions to those projects?

Almost everything I build stands on open source: React, Angular, React Native, Redux, RxJS, Webpack, Jest, Cypress and many more. I value the open-source model because it makes software better through transparency. Anyone can read the code, find the bug and propose the fix, and good ideas spread across the industry instead of staying locked inside one company. Open design systems are a good example. Spectrum's accessibility work, which I use daily, benefits everyone who studies or adopts it, and Vanilla can do the same for open-source projects that need a solid, accessible UI layer.

I also think open source needs sustainable maintainership: clear governance, good documentation, a welcoming contribution process, and companies that pay engineers to maintain what they depend on. Canonical's model is a strong example of that.

[CONFIRM: describe your own open-source activity honestly, e.g. issues reported, pull requests, public repos on GitHub. If you have not been a maintainer, say so directly, e.g. "I have not yet been an open-source maintainer; most of my work has been on private client codebases. Working in the open is one of the main reasons I am applying to Canonical."]

What software products have you yourself led which shipped many releases to multiple customers? What was your role?

- **Adobe Workfront – Document Management (2024–present), Dev Lead.** Workfront is used by many of the world's largest brands, and Document Management ships continuously to all of them. I drive planning and delivery, unblock engineers, review code, and keep developers, QA and design aligned. I am the first point of contact when a critical production issue needs diagnosing. *[CONFIRM: release cadence, e.g. "releases every two weeks"]*
- **Fiserv payments application (2022–2023), front-end owner from the first commit.** I designed the architecture, set up the Angular workspace, implemented the core features and multi-language support, and set the patterns the team followed. *[CONFIRM: how many releases/clients or markets it shipped to]*
- **My React Native crypto app (2025–present), sole owner** of design, architecture, build automation and releases through EAS/TestFlight. *[CONFIRM: is it public in the stores? number of releases or users, if any]*

What is your most senior role in a software engineering organisation?

My current role: **Senior UI Developer and Dev Lead** at Endava, leading the front-end team for Adobe Workfront's Document Management area. I joined Endava in 2017 as a UI Developer and was promoted from Mid-Level to Senior during the Cisco Webex engagement, in recognition of owning features from design handoff through end-to-end testing.

Describe your span of control and the diversity of products, functions and teams you led.

As Dev Lead I coordinate *[CONFIRM: number]* engineers and work daily with QA, design and product management, both on our team and across Adobe teams that share the same codebase and design system. *[CONFIRM: say whether you have formal line-management*

responsibility (performance reviews, hiring interviews) or a technical leadership role] Across my career I have worked in four product domains (payments, insurance, enterprise work management and advertising), with clients in different time zones and cultures, which taught me to adapt quickly to new teams and codebases. I mentor developers through code reviews, pairing and shared coding standards. *[CONFIRM: number of developers mentored, or an example of someone you helped grow]*

Outline your views on the role of an engineering manager in shaping a high-functioning team.

I think a good engineering manager mostly creates the conditions in which good engineers can do their best work:

- **Clarity.** Everyone should know what we are building, why it matters to the user, and what "done" means. Most team friction comes from unclear goals, not from bad engineers.
- **Psychological safety.** People must feel safe to say "I don't understand", "I made a mistake" or "I think this design is wrong". As a Dev Lead I try to show this myself, for example by admitting openly when I am not sure.
- **Removing blockers.** The manager's time is well spent unblocking others: getting a decision from product, clarifying an API with another team, or protecting focus time.
- **Growth.** Regular honest feedback, stretch tasks matched to each person, and mentoring. Teams stay strong when people can see themselves progressing.
- **High standards, shared by everyone.** Code review, testing and accessibility should be team habits, not personal preferences. The manager sets the bar and makes it easy to meet.
- **Healthy delivery rhythm.** Sustainable pace, small increments, and blameless retrospectives after incidents so the team learns instead of hiding problems.
- **Hiring and team composition.** Hiring for ability, curiosity and communication, and building a team with complementary strengths.

Outline your thoughts on performance in software engineering. How do you ensure that your product is fast?

Performance is a feature that users feel even when they cannot name it. My approach is to **measure, set budgets, and keep it fast continuously** instead of fixing it once.

- **Measure first:** Chrome DevTools performance profiles, the React Profiler, Lighthouse and Core Web Vitals (LCP, INP, CLS), and on mobile, frame rates and cold-start time. I don't optimise on assumptions.
- **Ship less code:** code splitting and lazy loading per route, tree-shaking, avoiding heavy dependencies, and watching bundle size in CI.
- **Render less:** correct memoisation boundaries, stable references, normalised state, selectors, and virtualised lists for large data sets. In React Native these techniques plus Hermes and Reanimated are what made 60fps interactions possible.
- **Fetch smarter:** pagination, caching and request de-duplication (e.g. Apollo cache), avoiding waterfalls, and good API design. A well-shaped REST endpoint is often the biggest performance improvement.

- **Perceived performance:** skeletons, optimistic updates and progressive rendering, so the interface responds immediately.
- **CSS and assets:** modern layout instead of JavaScript-driven layout, optimised images and fonts, and avoiding layout thrashing.
- **Guard against regressions:** performance budgets and checks in CI, so the product stays fast as the team adds features.

[CONFIRM: add one measurable example from your work, e.g. bundle size reduced by X%, load time from A to B]

Outline your thoughts on quality in software development. What practices are most effective in software teams to drive improvements in quality?

Quality is not a phase at the end. It is the result of many small habits that the whole team shares. The practices I have seen work best:

- **Thoughtful code review** focused on correctness, readability and design, not just style. Linters and formatters (ESLint, Prettier) should handle style automatically so humans can focus on what matters.
- **Strong typing:** TypeScript catches a whole class of bugs before they exist, especially at API boundaries.
- **A balanced test suite:** fast unit and component tests (Jest, Testing Library), and end-to-end tests (Cypress) for critical journeys. On Cisco Webex I covered critical user flows with Cypress, which gave the team confidence to refactor.
- **CI that blocks merges** on failing tests, lint or type errors, and small pull requests that are easy to review.
- **Design systems:** reusable, accessible, well-tested components mean each bug is fixed once, for every product.
- **Accessibility as part of the definition of done:** semantic HTML, keyboard navigation, screen-reader checks and contrast are quality issues, not extras.
- **Close collaboration with QA and design** from the start, so requirements and edge cases are clear before coding begins. In payments, edge cases are where the real bugs live.
- **Learning from incidents:** blameless post-mortems and turning every production bug into a regression test.
- **Paying down technical debt deliberately**, as I did at Mastercard, instead of letting it accumulate until it blocks delivery.

3. Education

How did you rank in your final year of high school in mathematics? Were you a top student? On what basis would you say that?

[CONFIRM: your final-year maths grade (Moldova uses a 1–10 scale, 10 is the highest), your position in class/school, and the evidence – e.g. "I had a 10/10 average in mathematics and was among the top 3 in my class; I represented my school at the district/national mathematics olympiad, where I placed ..."]

How did you rank in your final year of high school, in your home language? Were you a top student? On what basis would you say that?

My native languages are Romanian and Russian. *[CONFIRM: which was your language of instruction, your final-year grade and rank, and the evidence, e.g. olympiads, essay competitions, BAC result in that subject]*

Please state your high school graduation results or university entrance results, and explain the grading system used.

In Moldova, high school ends with the national Baccalaureate examination (BAC). Results are graded on a 1–10 scale, where 10 is the highest and 5 is the minimum pass. University admission is based mainly on the BAC average together with the school-year averages.

[CONFIRM: your BAC overall average and subject grades (e.g. Mathematics X, Romanian X, ...), your school graduation average, and whether you were admitted to university on a state-funded (budget) place, which is competitive and based on these grades]

Can you make a case that you are in the top 5% in your academic year, or top 1%, or even higher? If so please outline that case.

[CONFIRM: only include what you can support.] My case rests on the following:

- **Gold Medal for academic excellence.** *[CONFIRM: was it awarded at high school graduation or at university? In Moldova the high-school Gold Medal is given only to graduates with top grades in all subjects across the final years – state the criteria and how many students in your school/year received it.]*
- **Erasmus Mundus exchange in Prague.** *[CONFIRM: host university, duration, and how competitive the selection was, e.g. "one of N students selected from the faculty"]*
- *[CONFIRM: any olympiad placements, scholarships, ranking in your university cohort, or other standardised results]*

What sort of high school student were you? Outside of class, what were your interests and hobbies? What would your high school peers remember you for?

[CONFIRM: describe yourself in your own words – e.g. disciplined/curious, favourite subjects, hobbies (sport, music, art, languages, programming), clubs, and what classmates would remember you for]

Which university and degree did you choose? What other universities did you consider, and why did you select that one?

I studied for a **Bachelor's degree in Information Technology at the Technical University of Moldova (2012–2016)**, the country's leading technical university and the main source of engineers for Moldova's IT industry. *[CONFIRM: other universities you considered (e.g. Moldova State University, universities in Romania) and your real reasons for choosing UTM – e.g. reputation in engineering, practical programme, links with IT companies, scholarship]*

Overall, what was your degree result and how did that reflect on your ability? Please help us understand the grading system for your results.

Moldovan universities grade on the same 1–10 scale (10 highest, 5 minimum pass). *[CONFIRM: your overall average, licence (thesis) grade and thesis topic, and your rank in the cohort if known, plus whether you studied on a merit scholarship]* During my studies I also took part in the Erasmus Mundus exchange in Prague, which gave me experience of a different academic system and an international environment early on.

During all of your education years, from high school to university, can you describe any achievements that were truly exceptional?

- Gold Medal for academic excellence. *[CONFIRM: details as above]*
- Selection for the Erasmus Mundus exchange programme in Prague. *[CONFIRM: details]*
- *[CONFIRM: any other exceptional achievement, e.g. olympiads, competitions, research, awards]*

What leadership roles did you take on during your education? Did you conceive of, and drive to completion, any initiatives outside of your required classwork?

[CONFIRM: e.g. class president, student council, group leader at university, organising events, volunteering, starting a club or project. If none, describe initiative you took in your own learning, e.g. learning web development on your own before your first job.]

4. Context

Outline your thoughts on the mission of Canonical. What is it about the company's purpose and goals which is most appealing to you? What is risky or unappealing? Are there any elements of the company goals that you are unsure about?

As I understand it, Canonical's mission is to make open-source software secure, reliable and easy to use for everyone, from a single developer's laptop to the largest clouds, data centres and edge devices, and to fund that work sustainably through enterprise support and services. Ubuntu is the visible part of the mission. Behind it are long-term security maintenance, cloud and Kubernetes tooling, IoT, and the web products that let people discover, install and manage all of this.

What appeals to me most is that the work is open and has very wide reach. Improvements made here are used by millions of people and organisations, not only one client. As a front-end engineer I find the Vanilla Framework especially meaningful: a shared, accessible presentation layer that brings consistency to Canonical's products and can be adopted by other open-source projects that often have no designers or front-end specialists. Accessibility done well in a shared toolkit spreads to every product built on it. I also like that Canonical treats design as engineering's partner.

What I see as risky: Canonical competes against companies with much larger budgets while also giving much of its work away for free. That requires very careful focus. Some product decisions (Snap, for example) have caused friction with parts of the community, and

being both a community steward and a commercial vendor is a difficult balance. The globally distributed, remote-first model with a demanding bar is a great strength, but it also needs excellent written communication and discipline. I see that as a challenge I would enjoy rather than a negative.

What I am unsure about: the long-term role of Flutter. It is a promising way to unify desktop and web, but it depends heavily on Google's continued investment, and Flutter on the web still has trade-offs in accessibility, SEO and bundle size compared with React. I would like to understand how Canonical decides which products should be built in Flutter and which stay in React.

Who are competitors to Canonical, and what does Canonical need to change to be a more effective competitor?

- **Enterprise Linux:** Red Hat (IBM) with RHEL and OpenShift, SUSE, and community rebuilds such as Rocky Linux and AlmaLinux.
- **Cloud providers' own platforms:** Amazon Linux, Azure Linux, Google's Container-Optimized OS, and managed Kubernetes services that reduce the need for a separately supported OS.
- **Developer desktop:** macOS and Windows with WSL, which, ironically, often runs Ubuntu.
- **Private cloud and infrastructure:** VMware (Broadcom) and Nutanix, and Red Hat OpenShift for Kubernetes.
- **Secure container images and supply chain:** companies like Chainguard and Docker's hardened images.

To compete more effectively, I think Canonical should:

- Make the **web experience** of its products as polished and self-service as the best SaaS: clear docs, fast and accessible dashboards, simple trial and purchase flows. Enterprise buyers often judge a platform by its interfaces first. This is where web engineering has direct commercial impact.
- Communicate its **security and long-term support value** more clearly, especially in the current climate around Red Hat's source policies and VMware's licensing changes, which are opportunities to win customers.
- Keep strengthening **community trust** through openness about decisions such as Snap, so developers stay advocates.
- Grow the **Vanilla Framework** outside Canonical, with strong documentation, React components and examples, so that it becomes a recognised open-source design system and a channel to developers.

What would you most want to change about Canonical?

From the outside, I would most like to see Canonical's design and front-end work become more visible to the wider developer community. Vanilla is a strong project, but it is much less known than Material UI, Carbon or Spectrum. More public documentation, examples, talks and contributions to accessibility standards would attract more open-source projects to it and help hiring too. I would be glad to contribute to that. I am aware this is an outside view,

and I would first want to understand the reasons behind current priorities before suggesting changes.

What gets you most excited about this role?

- **Working in the open.** After nine years on private client codebases, the chance to build software that anyone can use, read and improve is the main reason I am applying.
- **Design systems and accessibility at scale.** I have worked on top of React Spectrum and other design systems for years. Contributing to Vanilla and helping more projects build accessible, responsive interfaces is exactly the kind of work I enjoy most.
- **React plus Flutter.** My recent React Native work showed me the value of one codebase across platforms. Growing into Flutter, with a real opportunity to unify desktop and web, is a challenge I am excited about.
- **Full-stack ownership and well-designed APIs.** I want to work in a place where front-end engineers help design the REST APIs and sometimes build the back-end service as well.
- **The people.** A global, high-bar team of engineers and designers from whom I can learn, and to whom I can bring my experience of leading and mentoring.